HypArr

Computations with hyperplane arrangements

0.1

19 March 2026

Paul Mücksch

Paul Mücksch
Email: paul.muecksch+uni@gmail.com
Homepage: https://paulmuecksch.github.io
Address: Welfengarten 1, 30167 Hannover, Germany

Contents

1 Introduction **3**

2 Hyperplane arrangements **4**
2.1 Attributes 4
2.2 Global methods 5
2.3 Constructors 7

3 Milnor fibers **8**
3.1 Complexes 8

4 Oriented Matroids **10**
4.1 Attributes 10
4.2 Properties 13
4.3 Global methods 14
4.4 Constructors 15

Index **16**

Chapter 1

Introduction

HypArr is a package for computations with hyperplane arrangements, oriented matroids, and their topological invariants, in particular Milnor fibers and complements.

Chapter 2

Hyperplane arrangements

2.1 Attributes

2.1.1 Dimension (for IsHyperplaneArrangement)

▷ `Dimension(A)` (attribute)

Returns: a non-negative integer.

Returns the dimension of the ambient vector space in which the hyperplane arrangement `A` is defined.

2.1.2 Roots (for IsHyperplaneArrangement)

▷ `Roots(A)` (attribute)

Returns: A list of vectors defining the hyperplanes of `A`.

Returns the list of defining linear forms (also called *roots*) of the hyperplanes in the arrangement $\mathscr{A}$. Each vector represents a hyperplane by the linear equation

$$r_1x_1 + \cdots + r_nx_n = 0.$$

2.1.3 IntersectionLattice (for IsHyperplaneArrangement)

▷ `IntersectionLattice(A)` (attribute)

Returns: A list of lists describing the intersections of `A`.

Computes the intersection lattice of the hyperplane arrangement $\mathscr{A}$.

The lattice is represented level-by-level. The entry $L[k]$ contains all intersections of rank k.

Each lattice element is stored as a list of indices referring to hyperplanes in `Roots(A)`.

For example, the list `[1,3]` represents the intersection of the first and third hyperplane of the arrangement.

The algorithm incrementally constructs the lattice by adding hyperplanes one by one.

Example

```
gap> A := HyperplaneArrangement([[1,0],[0,1],[1,1]]);
gap> IntersectionLattice(A);
[ [ [ 1 ], [ 2 ], [ 3 ] ],
  [ [ 1, 2 ], [ 1, 3 ], [ 2, 3 ] ],
  [ [ 1, 2, 3 ] ] ]
```

2.1.4 CharPoly (for IsHyperplaneArrangement)

▷ `CharPoly(A)` (attribute)

Returns: a univariate polynomial with integral coefficients.

Returns $\chi_{\mathscr{A}}$ the *characterisitc polynomial* of the arrangement $\mathscr{A}$.

Example
```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> CharPoly(A);
t^3-6*t^2+11*t-6
```

2.1.5 MSetInvL (for IsHyperplaneArrangement)

▷ `MSetInvL(A)` (attribute)

Returns: A list encoding multiset invariants of the intersection lattice.

Computes multiset invariants of the intersection lattice of the hyperplane arrangement `A`.

For each level of the lattice, the function records how many intersections occur with a given number of defining hyperplanes.

These invariants can be used to compare arrangements or detect combinatorial similarities between intersection lattices.

2.1.6 IsReal (for IsHyperplaneArrangement)

▷ `IsReal(A)` (property)

Returns: true or false.

Determines, if the hyperplane arrangement is defined over the reals, i.e., if the entries of the defining linear forms are real.

2.1.7 SalvettiComplex (for IsHyperplaneArrangement)

▷ `SalvettiComplex(A)` (attribute)

Returns: FacePoset

Constructs the Salvetti Complex. If `IsReal(A)` is false, then it returns `fail`.

2.2 Global methods

2.2.1 DrawLatex3Arr

▷ `DrawLatex3Arr(A[, ps[, ip[, Hind[, disthv[, MarkHs]]]]])` (function)

Returns: A string.

Generates tikz-code for a nice projective picture of the real 3-arrangement.

The example below will look as follows (only in pdf).

Example
```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> Print(DrawLatex3Arr(A));
\begin{tikzpicture}[scale=1.0]
\draw (-2.828,2.828) -- (2.828,-2.828);  % H_1
\draw (2.828,2.828) -- (-2.828,-2.828);  % H_2
```

```
\draw (-1.,3.872) -- (-1.,-3.872);  % H_3
\draw (1.,3.872) -- (1.,-3.872);  % H_4
\draw (3.872,-1.) -- (-3.872,-1.);  % H_5
\draw (3.872,1.) -- (-3.872,1.);  % H_6

\end{tikzpicture}
```

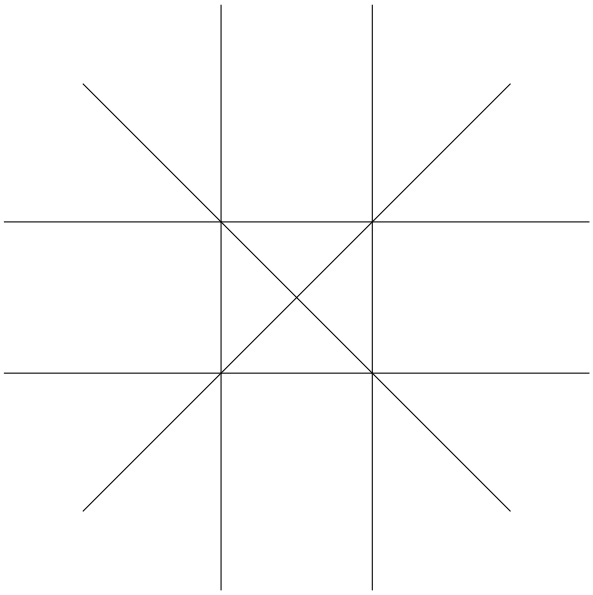

Figure 1: An example of a picture produced by `DrawLatex3Arr`.

2.2.2 AGpql

▷ `AGpql(p, q, l)` (function)

Returns: A hyperplane arrangement.

Constructs the reflection arrangement associated to the monomial complex reflection group $G(p,q,l)$. The hyperplanes are defined by equations of the form

$$x_i = \zeta^k x_j$$

where ζ is a primitive p-th root of unity.

Example

```
gap> A := AGpql(2,1,3);
 <HyperplaneArrangement: 9 hyperplanes in 3-space>
gap> Roots(A);
[ [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ],
  [ 1, 1, 0 ], [ 1, -1, 0 ], [ 1, 0, 1 ],
  [ 1, 0, -1 ], [ 0, 1, 1 ], [ 0, 1, -1 ] ]
```

2.2.3 HArrResHind

▷ `HArrResHind(A)` (function)

Returns: A hyperplane arrangement.

Computes the *restriction* of the arrangement `A` to the k-th hyperplane of `Roots(A)`.

2.2.4 HArrResX

▷ `HArrResX(A, S)` (function)

Returns: A hyperplane arrangement.

Computes the *restriction* of the arrangement `A` to the subspace spanned by the vectors in `S`.

2.2.5 Essentialization

▷ `Essentialization(A)` (function)

Returns: A hyperplane arrangement.

Computes the *essentialization* of the hyperplane arrangement An arrangement is called essential if the intersection of all its hyperplanes is the origin. If this is not the case, the function restricts the arrangement to a complementary subspace so that the resulting arrangement becomes essential.

2.3 Constructors

2.3.1 HyperplaneArrangement (for IsList)

▷ `HyperplaneArrangement(r)` (operation)

Returns: A hyperplane arrangement

Constructs a hyperplane arrangement from a list `R` of vectors representing defining linear forms of hyperplanes.

Each vector $r = [r_1, \ldots, r_n]$ corresponds to the hyperplane

$$r_1 x_1 + \cdots + r_n x_n = 0.$$

The vectors must lie in the same vector space.

Linearly dependent defining forms are removed automatically, so the resulting arrangement only stores pairwise linearly independent hyperplanes.

Example

```
gap> A := HyperplaneArrangement([[1,0,0],[0,1,0],[0,0,1]]);
<HyperplaneArrangement: 3 hyperplanes in 3-space>
gap> Roots(A);
[ [ 1, 0, 0 ], [ 0, 1, 0 ], [ 0, 0, 1 ] ]
```

Chapter 3

Milnor fibers

The Milnor fiber is the typical fiber f^{-1} of the Milnor fibration

$$f : \mathbb{C}^{\ell} \setminus \cup_{H \in \mathscr{A}} H \to \mathbb{C}^{\times}, z \mapsto \prod_{H \in \mathscr{A}} \alpha_H(z)$$

of the hyperplane arrangement $\mathscr{A}$, where α_H are defining linear forms.

The following describes functions to construct regular cell complex having the homotopy type of the Milnor fiber and thus enables computation of homotopy invaraints using e.g. the HAP package.

3.1 Complexes

3.1.1 MilnorFiberComplex (for IsOrientedMatroid)

▷ `MilnorFiberComplex(OM)` (attribute)

Returns: FacePoset

Computes the face poset of the regular cell complex of the combinatorial Milnor fiber of the oriented matroid `OM`.

Example

```
gap> O:=OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> MCpx:=MilnorFiberComplex(O);
<FacePoset of dimension 1 with f-vector [ 3, 6 ]>
```

3.1.2 MilnorFiberComplex (for IsHyperplaneArrangement)

▷ `MilnorFiberComplex(A)` (attribute)

Returns: FacePoset

Computes the face poset of the regular cell complex having the homotopy type of Milnor fiber of the arrangement `A`.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> MCpx:=MilnorFiberComplex(A);
<FacePoset of dimension 2 with f-vector [ 12, 60, 60 ]>
```

3.1.3 FPtoCWCpx

▷ `FPtoCWCpx(FP)` (function)

Returns: HAP CW Complex

Converts a HYPARR `FacePoset` into a HAP `RegularCWComplex` for further computations of topological invariants.

Example

```
gap> A:=AGpql(2,2,3);
<HyperplaneArrangement: 6 hyperplanes in 3-space>
gap> MCpx:=MilnorFiberComplex(A);
<FacePoset of dimension 2 with f-vector [ 12, 60, 60 ]>
gap> MCW := FPtoCWCpx(MCpx);
Regular CW-complex of dimension 2
gap> Homology(MCW,0);
[ 0 ]
gap> Homology(MCW,1);
[ 0, 0, 0, 0, 0, 0, 0 ]
```

Chapter 4

Oriented Matroids

Oriented matroids are a abstraction of real hyperplane arrangements. The following describes functions to handle oriented matroids and compute associated cell complexes and invariants.

4.1 Attributes

4.1.1 OMRank (for IsOrientedMatroid)

▷ `OMRank(OM)` (attribute)

Returns: An integer

Returns the rank of the oriented matroid *OM*.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMRank(OM);
3
```

4.1.2 OMGroundset (for IsOrientedMatroid)

▷ `OMGroundset(OM)` (attribute)

Returns: A list

Returns the ground set of the oriented matroid OM.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMGroundset(OM);
[ 1, 2, 3, 4, 5, 6 ]
```

4.1.3 OMLForms (for IsOrientedMatroid)

▷ `OMLForms(OM)` (attribute)

Returns: A list of linear forms or `fail`

Returns the list of linear forms associated with the oriented matroid OM. If the internal component `OM!.lforms` is not bound, `fail` is returned.

Example
```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMLForms(O);
[ [ 1, 1, 0 ], [ 1, -1, 0 ], [ 1, 0, 1 ],
  [ 1, 0, -1 ], [ 0, 1, 1 ], [ 0, 1, -1 ] ]
```

4.1.4 OMGeomLattice (for IsOrientedMatroid)

▷ `OMGeomLattice(OM)` (attribute)

Returns: A list of lists or `fail`

Computes the geometric lattice associated with the oriented matroid OM. If OM is linear (as determined by `OMIsLinear(OM)`), the lattice is computed as the intersection lattice of the hyperplane arrangement given by `OMLForms(OM)`, stored in the internal component `OM!.lattice`, and returned. If OM is not linear, `fail` is returned.

Example
```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
gap> OMGeomLattice(OM);
[ [ [ 1 ], [ 2 ], [ 3 ], [ 4 ], [ 5 ], [ 6 ] ],
[ [ 1, 2 ], [ 1, 3, 6 ], [ 2, 3, 5 ], [ 1, 4, 5 ],
  [ 2, 4, 6 ], [ 3, 4 ], [ 5, 6 ] ],
  [ [ 1, 2, 3, 4, 5, 6 ] ] ]
```

4.1.5 OMCocircuits (for IsOrientedMatroid)

▷ `OMCocircuits(OM)` (attribute)

Returns: A list

Computes the (signed) cocircuits of the oriented matroid OM. The cocircuits are derived from the geometric lattice of OM (via `OMGeomLattice(OM)`) and the linear forms (from `OMLForms(OM)`). The result is a list of signed vectors representing all cocircuits of OM.

Example
```
gap> O:=OrientedMatroid([[1,0],[0,1],[1,1]]); OMCocircuits(O);
<OrientedMatroid: 3 elements, rank 2>
[ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
  [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ]
```

4.1.6 TopeGraph (for IsOrientedMatroid)

▷ `TopeGraph(OM)` (attribute)

Returns: A graph

Computes the tope graph of the oriented matroid OM. The topes are obtained as the first set of covectors from `OMCovectors(OM)`. Two topes are connected by an edge if their separating set has at most one element.

Example
```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> G := ShallowCopy(TopeGraph(O));;
gap> Vertices(G);
```

```
[ 1 .. 6 ]
gap> IsSimpleGraph(G);
true
gap> UndirectedEdges(G);
[ [ 1, 3 ], [ 1, 5 ], [ 2, 4 ], [ 2, 6 ], [ 3, 6 ], [ 4, 5 ] ]
```

4.1.7 OMCovectors (for IsOrientedMatroid)

▷ `OMCovectors(OM)` (attribute)

Returns: A list of lists

Computes all covectors of the oriented matroid OM. The result is returned as a reversed list of lists, where each inner list is a covector represented as a signed vector.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> OMCovectors(O);
[ [ [ -1, 1, -1 ], [ 1, -1, 1 ], [ -1, -1, -1 ],
    [ 1, 1, 1 ], [ -1, 1, 1 ], [ 1, -1, -1 ] ],
  [ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
    [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ],
  [ [ 0, 0, 0 ] ] ]
```

4.1.8 SalvettiComplex (for IsOrientedMatroid)

▷ `SalvettiComplex(OM)` (attribute)

Returns: A Salvetti complex object

Constructs the Salvetti complex associated with the oriented matroid OM.

The construction proceeds as follows:

1. Computes the covectors of OM via `OMCovectors(OM)`.
2. Uses the topes and the lower order ideals of the covector poset to form the cells of the complex.
3. Defines an order function on the cells to capture the face poset structure, using `OMOperation` and `OrderCovec`.
4. Returns an object of type `FacePosetFamily` representing the Salvetti complex, with its ground set stored in `grgroundset` and the order function in `orderfunction`.

Example

```
gap> O := OrientedMatroid([[1,0],[0,1],[1,1]]);
<OrientedMatroid: 3 elements, rank 2>
gap> SalvettiComplex(O);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
```

4.1.9 FPGroundset (for IsFacePoset)

▷ `FPGroundset(FP)` (attribute)

Returns: A list

Returns the ground set of the face poset FP.

Example

```
gap> S := SalvettiComplex(O);
<FacePoset of dimension 2 with f-vector [ 6, 12, 6 ]>
```

```
gap> FPGroundset(S);
[
  [
    [ [ -1, 1, -1 ], [ -1, 1, -1 ] ],
    [ [ 1, -1, 1 ], [ 1, -1, 1 ] ],
    [ [ -1, -1, -1 ], [ -1, -1, -1 ] ],
    [ [ 1, 1, 1 ], [ 1, 1, 1 ] ],
    [ [ -1, 1, 1 ], [ -1, 1, 1 ] ],
    [ [ 1, -1, -1 ], [ 1, -1, -1 ] ]
  ],
  [
    [ [ -1, 0, -1 ], [ -1, 1, -1 ] ],
    [ [ -1, 1, 0 ], [ -1, 1, -1 ] ],
    [ [ 1, 0, 1 ], [ 1, -1, 1 ] ],
    [ [ 1, -1, 0 ], [ 1, -1, 1 ] ],
    [ [ 0, -1, -1 ], [ -1, -1, -1 ] ],
    [ [ -1, 0, -1 ], [ -1, -1, -1 ] ],
    [ [ 0, 1, 1 ], [ 1, 1, 1 ] ],
    [ [ 1, 0, 1 ], [ 1, 1, 1 ] ],
    [ [ 0, 1, 1 ], [ -1, 1, 1 ] ],
    [ [ -1, 1, 0 ], [ -1, 1, 1 ] ],
    [ [ 0, -1, -1 ], [ 1, -1, -1 ] ],
    [ [ 1, -1, 0 ], [ 1, -1, -1 ] ]
  ],
  [
    [ [ 0, 0, 0 ], [ -1, 1, -1 ] ],
    [ [ 0, 0, 0 ], [ 1, -1, 1 ] ],
    [ [ 0, 0, 0 ], [ -1, -1, -1 ] ],
    [ [ 0, 0, 0 ], [ 1, 1, 1 ] ],
    [ [ 0, 0, 0 ], [ -1, 1, 1 ] ],
    [ [ 0, 0, 0 ], [ 1, -1, -1 ] ]
  ]
]
```

4.1.10 FPOrder (for IsFacePoset)

▷ `FPOrder(FP)` (attribute)

Returns: A function

Returns the order function of the face poset FP.

4.2 Properties

4.2.1 OMIsLinear (for IsOrientedMatroid)

▷ `OMIsLinear(OM)` (property)

Returns: true or false

Returns whether the oriented matroid OM is linear. This is the case if the internal component `OM!.lforms` is bound.

Example

```
gap> A := AGpql(2,2,3); OM := OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
```

```
gap> OMIsLinear(OM);
true
```

4.3 Global methods

4.3.1 OMOperation

▷ `OMOperation(sv1, sv2)` (function)

Returns: A list

Performs the oriented matroid operation on two signed vectors `sv1` and `sv2`. For each position `i`, if `sv1[i]` is nonzero, it is used; otherwise `sv2[i]` is used. Returns a new signed vector representing the result of the operation.

Example

```
gap> CVs:=OMCovectors(O)[2];
[ [ 0, 1, 1 ], [ 0, -1, -1 ], [ 1, 0, 1 ],
  [ -1, 0, -1 ], [ -1, 1, 0 ], [ 1, -1, 0 ] ]
gap> sv1 := CVs[3]; sv2 := CVs[5];
[ 1, 0, 1 ]
[ -1, 1, 0 ]
gap> OMOperation(sv1, sv2);
[ 1, 1, 1 ]
gap> OMOperation(sv2, sv1);
[ -1, 1, 1 ]
```

4.3.2 OrderCovec

▷ `OrderCovec()` (function)

Computes

4.3.3 LowerOrderIdeal

▷ `LowerOrderIdeal()` (function)

Computes

4.3.4 IsOMEquiv

▷ `IsOMEquiv()` (function)

Computes

4.3.5 SeparatingSet

▷ `SeparatingSet()` (function)

Computes

4.4 Constructors

4.4.1 OrientedMatroid (for IsList)

▷ `OrientedMatroid(r, or, A)` (operation)

Returns: An oriented matroid OM.

Constructs am oriented matroid from a list `r` of vectors representing defining linear forms of real hyperplanes or a real hyperplane arrangement `A`.

Each vector $r = [r_1, \ldots, r_n]$ corresponds to the linear form

$$r_1 x_1 + \cdots + r_n x_n = 0.$$

The vectors must lie in the same real vector space.

Example

```
gap> A:=AGpql(2,2,3); OM:=OrientedMatroid(Roots(A));
<OrientedMatroid: 6 elements, rank 3>
gap> A:=AGpql(2,2,3); OM:=OrientedMatroid(A);
<OrientedMatroid: 6 elements, rank 3>
```

Index

AGpql, 6

CharPoly
  for IsHyperplaneArrangement, 5

Dimension
  for IsHyperplaneArrangement, 4
DrawLatex3Arr, 5

Essentialization, 7

FPGroundset
  for IsFacePoset, 12
FPOrder
  for IsFacePoset, 13
FPtoCWCpx, 9

HArrResHind, 6
HArrResX, 7
HyperplaneArrangement
  for IsList, 7

IntersectionLattice
  for IsHyperplaneArrangement, 4
IsOMEquiv, 14
IsReal
  for IsHyperplaneArrangement, 5

LowerOrderIdeal, 14

MilnorFiberComplex
  for IsHyperplaneArrangement, 8
  for IsOrientedMatroid, 8
MSetInvL
  for IsHyperplaneArrangement, 5

OMCocircuits
  for IsOrientedMatroid, 11
OMCovectors
  for IsOrientedMatroid, 12
OMGeomLattice
  for IsOrientedMatroid, 11
OMGroundset
  for IsOrientedMatroid, 10
OMIsLinear
  for IsOrientedMatroid, 13
OMLForms
  for IsOrientedMatroid, 10
OMOperation, 14
OMRank
  for IsOrientedMatroid, 10
OrderCovec, 14
OrientedMatroid
  for IsList, 15

Roots
  for IsHyperplaneArrangement, 4

SalvettiComplex
  for IsHyperplaneArrangement, 5
  for IsOrientedMatroid, 12
SeparatingSet, 14

TopeGraph
  for IsOrientedMatroid, 11